



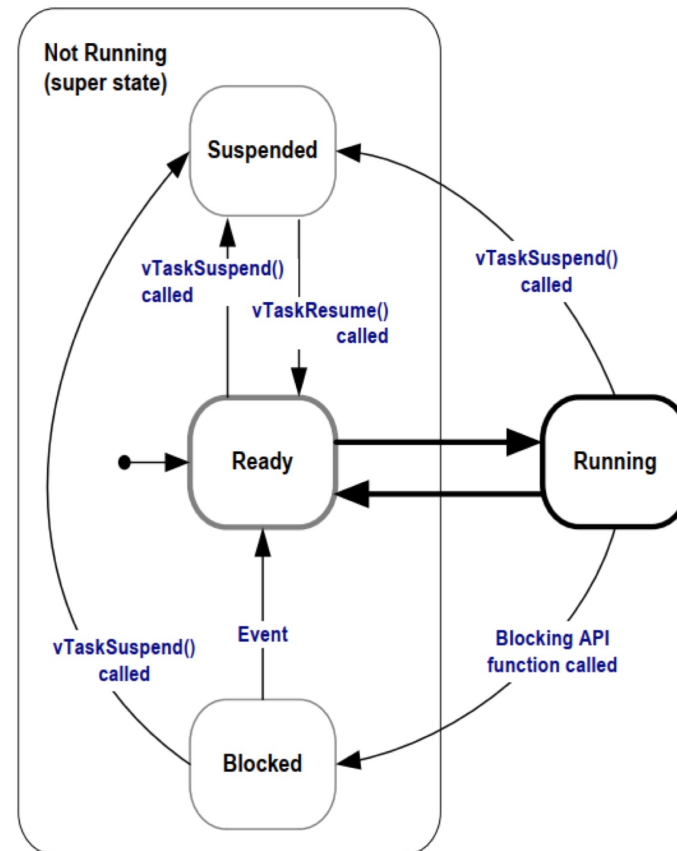
universidade de aveiro

# Dispositivos Conectados

## Operação de um sistema embutido

Paulo C. Bartolomeu

## Máquina de estados de uma tarefa



## Máquina de estados de uma tarefa

- Uma tarefa que está realmente em execução (usando tempo de processamento) está no estado **Running** (Em execução). Num processador de núcleo único, só pode haver uma tarefa no estado **Running** em qualquer momento.
- As tarefas que não estão realmente em execução, mas que não estão no estado **Blocked** (Bloqueado) ou **Suspended** (Suspendido), estão no estado **Ready** (Pronto).
- As tarefas no estado **Ready** estão disponíveis para serem selecionadas pelo escalonador como a tarefa a entrar no estado **Running**. O escalonador sempre escolherá a tarefa no estado Pronto com a prioridade mais alta para entrar no estado Em execução.
- As tarefas podem esperar no estado **Bloqueado** por um evento e são automaticamente movidas de volta para o estado **Pronto** quando o evento ocorre.
- Eventos temporais ocorrem num momento específico, por exemplo, quando um tempo de bloqueio expira, e são normalmente usados para implementar um comportamento periódico ou de tempo limite.
- Os eventos de sincronização ocorrem quando uma tarefa ou rotina de serviço de interrupção envia informações usando uma notificação de tarefa, fila, grupo de eventos, buffer de mensagens, buffer de fluxo ou um dos muitos tipos de semáforo. Geralmente são usados para sinalizar atividades assíncronas, como dados chegando a um periférico.



## Máquina de estados de uma tarefa

- Deve haver sempre pelo menos uma tarefa que possa entrar no estado **Running**. Para garantir que isso aconteça, o escalonador cria automaticamente uma tarefa *Idle* quando `vTaskStartScheduler()` é chamado.
- A tarefa *idle* (ociosa) faz muito pouco além de ficar num ciclo, portanto está sempre pronta para ser executada.
- A tarefa ociosa tem a prioridade mais baixa possível (prioridade zero), para garantir que nunca impeça uma tarefa de aplicação com prioridade mais alta de entrar no estado **Running**.
  - No entanto, nada impede que os *developers* de aplicações criem tarefas com a prioridade da tarefa ociosa e, portanto, que a compartilhem, se desejarem.

## Algoritmos de escalonamento de tarefas

- O algoritmo de escalonamento é a rotina de *software* que decide qual tarefa do estado **Pronto** transitará para o estado **Em Execução**.
- Em todas as configurações possíveis de núcleo único, o escalonador do FreeRTOS seleciona tarefas que partilham uma prioridade vez à vez. Essa política de "assumir a ordem" é frequentemente chamada de "Escalonamento Round Robin".
- Um algoritmo de escalonamento Round Robin não garante que o tempo seja partilhado igualmente entre tarefas de igual prioridade, apenas que tarefas no estado **Pronto** com a mesma prioridade entrem no estado **Em Execução** vez à vez.

## Configuração do algoritmos de escalonamento

- Preemptive: Algoritmos de escalonamento preemptivo "preemptarão" imediatamente a tarefa em estado de **Execução** se uma tarefa com prioridade maior que a tarefa em estado de **Execução** entrar no estado **Pronto**. Ser preemptivo significa ser movido involuntariamente do estado de **Execução** para o estado **Pronto** (sem ceder ou bloquear explicitamente) para permitir que uma tarefa diferente entre no estado de **Execução**. A preempção de tarefas pode ocorrer a qualquer momento, não apenas na interrupção de *tick* do RTOS.

| Scheduling Algorithm            | Prioritized | configUSE_PREEMPTION | configUSE_TIME_SLICING |
|---------------------------------|-------------|----------------------|------------------------|
| Preemptive With Time Slicing    | Yes         | 1                    | 1                      |
| Preemptive Without Time Slicing | Yes         | 1                    | 0                      |
| Co-Operative                    | No          | 0                    | Any                    |

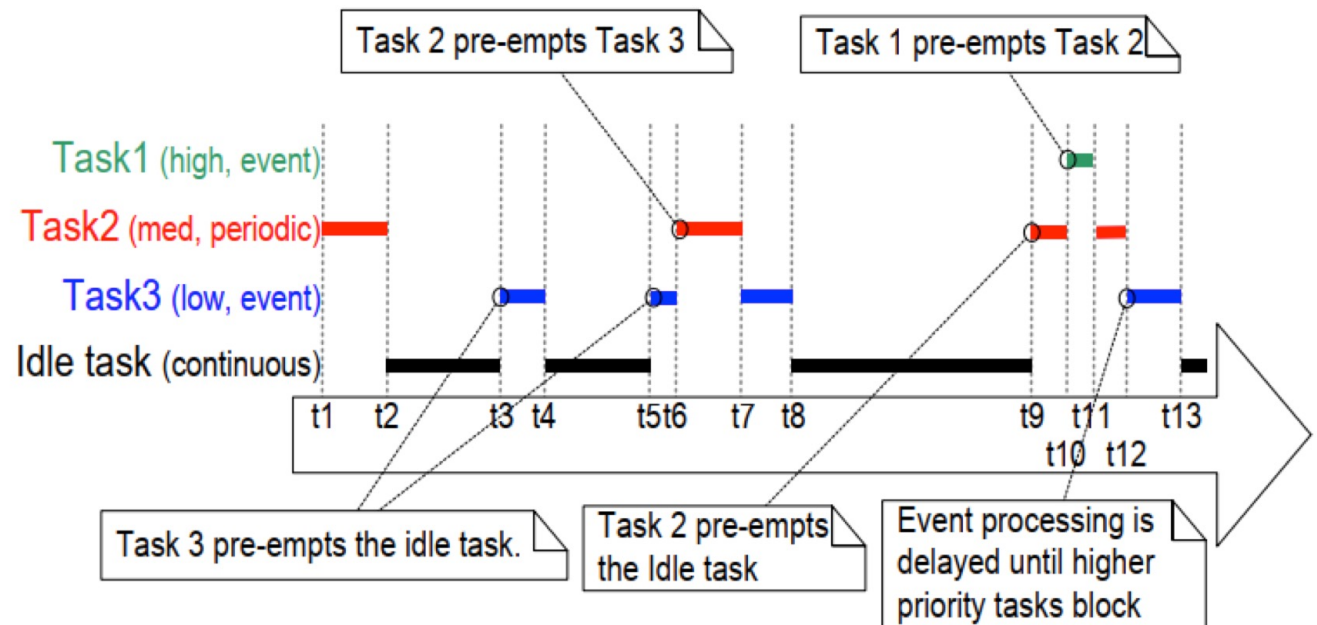
## Configuração do algoritmos de escalonamento

- Time Slicing: O fatiamento (*slicing*) de tempo é usado para partilhar o tempo de processamento entre tarefas da mesma prioridade, mesmo quando as tarefas não cedem ou entram explicitamente no estado **Bloqueado**. Algoritmos de escalonamento descritos como usando Fatiamento de Tempo selecionam uma nova tarefa para entrar no estado **Em Execução** ao final de cada fatia de tempo se houver outras tarefas no estado **Pronto** com a mesma prioridade que a tarefa **Em Execução**. Uma *time slice* é igual ao tempo entre duas interrupções de *tick* do RTOS.

| Scheduling Algorithm            | Prioritized | configUSE_PREEMPTION | configUSE_TIME_SLICING |
|---------------------------------|-------------|----------------------|------------------------|
| Preemptive With Time Slicing    | Yes         | 1                    | 1                      |
| Preemptive Without Time Slicing | Yes         | 1                    | 0                      |
| Co-Operative                    | No          | 0                    | Any                    |

## Algoritmo de agendamento preemptivo de prioridade fixa com time slicing

- Padrão de execução destacando a priorização e a preempção de tarefas numa aplicação hipotética na qual cada tarefa recebeu uma prioridade única.



## Algoritmo de agendamento preemptivo de prioridade fixa com time slicing

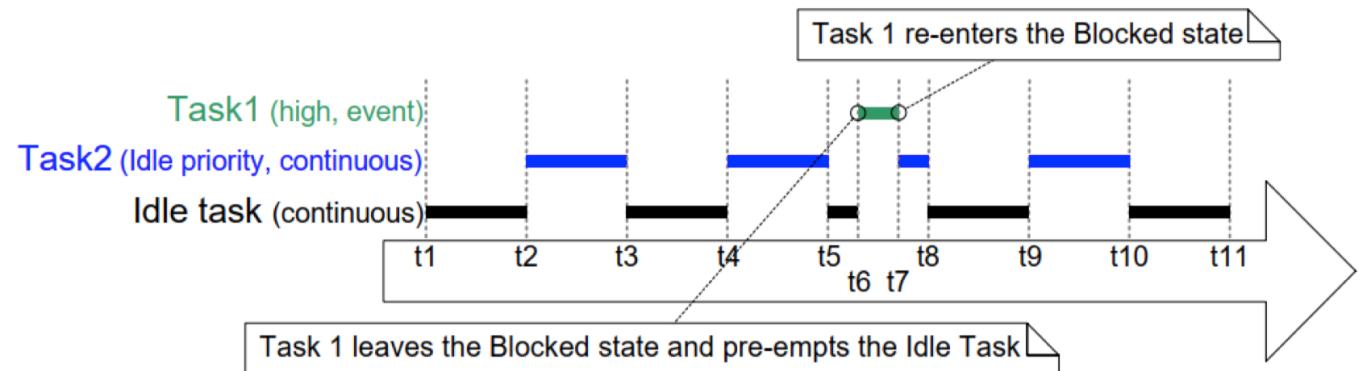
- **Idle task:** A tarefa *idle* é executada na prioridade mais baixa. É interrompida sempre que uma tarefa de prioridade mais alta entra no estado **Pronto**, por exemplo, nos momentos t3, t5 e t9.
- **Task 3:** A Tarefa 3 é uma tarefa orientada a eventos que é executada com uma prioridade relativamente baixa, mas acima da prioridade **Ociosa**.
  - Passa a maior parte do tempo no estado **Bloqueado**, aguardando o evento de interesse, transitando do estado **Bloqueado** para o estado **Pronto** sempre que o evento ocorre.
  - Todos os mecanismos de comunicação entre tarefas do FreeRTOS (notificações de tarefas, filas, semáforos, grupos de eventos, etc.) podem ser usados para sinalizar eventos e desbloquear tarefas dessa maneira.
  - Os eventos ocorrem nos instantes t3 e t5, e também em algum ponto entre t9 e t12. Os eventos que ocorrem nos instantes t3 e t5 são processados imediatamente porque, nesses instantes, a Tarefa 3 é a tarefa de maior prioridade que pode ser executada.
  - O evento que ocorre em algum ponto entre t9 e t12 não é processado até t12 porque, até então, as tarefas de maior prioridade, Tarefa 1 e Tarefa 2, ainda estão em execução. É somente no instante t12 que as Tarefas 1 e 2 estão no estado **Bloqueado**, tornando a Tarefa 3 a tarefa de maior prioridade no estado **Pronto**.

## Algoritmo de agendamento preemptivo de prioridade fixa com time slicing

- **Task 2:** A Tarefa 2 é uma tarefa periódica executada com uma prioridade acima da prioridade da Tarefa 3, mas abaixo da prioridade da Tarefa 1. O intervalo de tempo da tarefa significa que a Tarefa 2 deseja executar nos tempos t1, t6 e t9.
  - No instante t6, a Tarefa 3 está no estado **Em Execução**, mas a Tarefa 2 tem a prioridade relativa mais alta, então interrompe a Tarefa 3 e inicia a execução imediatamente.
  - A Tarefa 2 conclui seu processamento e retorna ao estado **Bloqueado** no tempo t7, momento em que a Tarefa 3 pode retornar ao estado **Em Execução** para concluir seu processamento. A própria Tarefa 3 bloqueia no tempo t8.
- **Task 1:** A Tarefa 1 também é uma tarefa orientada a eventos. Ela é executada com a maior prioridade de todas, podendo, portanto, sobrepor-se a qualquer outra tarefa no sistema.
  - O único evento da Tarefa 1 mostrado ocorre no instante t10, momento em que a Tarefa 1 sobrepõe-se à Tarefa 2. A Tarefa 2 só pode concluir seu processamento após a Tarefa 1 retornar ao estado **Bloqueado** no instante t11.

## Algoritmo de agendamento preemptivo de prioridade fixa com time slicing

- Padrão de execução destacando a priorização de tarefas e o fatiamento de tempo numa aplicação hipotética no qual duas tarefas são executadas com a mesma prioridade.



## Algoritmo de agendamento preemptivo de prioridade fixa com time slicing

- **Idle task** e **Task 2**: A tarefa *idle* e a tarefa 2 são tarefas de processamento contínuo e ambas têm prioridade 0 (a menor prioridade possível).
  - O escalonador aloca tempo de processamento para as tarefas de prioridade 0 somente quando não há tarefas de prioridade mais alta que possam ser executadas e compartilha o tempo alocado para as tarefas de prioridade 0 por meio de fatiamento de tempo.
  - Uma nova fatia de tempo inicia a cada interrupção de *tick*, o que na Figura 4.19 ocorre nos tempos t1, t2, t3, t4, t5, t8, t9, t10 e t11.
  - A tarefa *idle* e a tarefa 2 entram no estado de execução alternadamente, o que pode resultar em ambas as tarefas permanecerem no estado de execução por parte da mesma fatia de tempo, como ocorre entre os tempos t5 e t8.

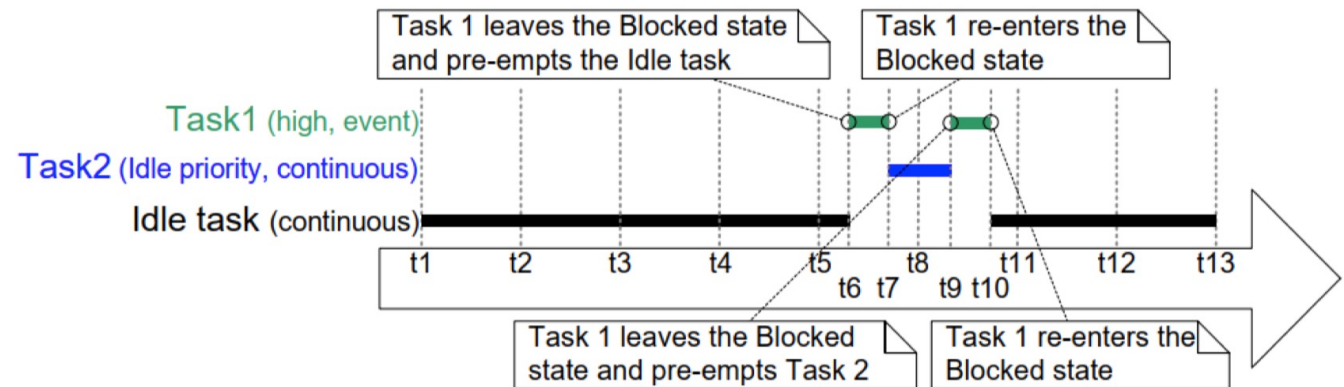
## Algoritmo de agendamento preemptivo de prioridade fixa com time slicing

- **Task 1:** A prioridade da Tarefa 1 é maior que a prioridade **Ociosa**.
  - A Tarefa 1 é uma tarefa orientada a eventos que passa a maior parte do tempo no estado **Bloqueado**, aguardando seu evento de interesse, transitando do estado **Bloqueado** para o estado **Pronto** sempre que o evento ocorre.
  - O evento de interesse ocorre no instante  $t_6$ . Em  $t_6$ , a Tarefa 1 torna-se a tarefa de maior prioridade que pode ser executada e, portanto, a Tarefa 1 antecipa a tarefa **Ociosa** no meio de um intervalo de tempo.
  - O processamento do evento é concluído no instante  $t_7$ , momento em que a Tarefa 1 retorna ao estado **Bloqueado**.

A Figura mostra a tarefa *idle* a partilhar o tempo de processamento com uma tarefa criada pelo *developer* da aplicação. Alocar tanto tempo de processamento para a tarefa *idle* pode não ser desejável se as tarefas prioritárias ociosas criadas pelo autor do aplicativo tiverem trabalho a fazer, mas a tarefa *idle* não.

## Algoritmo de agendamento preemptivo de prioridade fixa sem time slicing

- Padrão de execução que demonstra como tarefas de igual prioridade podem receber quantidades muito diferentes de tempo de processamento quando o fatiamento de tempo não é usado.



## Algoritmo de agendamento preemptivo de prioridade fixa sem time slicing

- O Agendamento Preemptivo priorizado sem fatiamento de tempo mantém os mesmos algoritmos de seleção e preempção de tarefas descritos antes, mas não utiliza fatiamento de tempo para compartilhar o tempo de processamento entre tarefas de igual prioridade.
- Como referido, se o fatiamento de tempo for utilizado e houver mais de uma tarefa em estado pronto com a prioridade mais alta capaz de ser executada, o escalonador seleciona uma nova tarefa para entrar no estado Em Execução durante cada interrupção de tique do RTOS (uma interrupção de tique marca o fim de uma fatia de tempo).
- **Se o fatiamento de tempo não for utilizado, o escalonador somente seleciona uma nova tarefa para entrar no estado Em Execução quando:**
  - Uma **tarefa de prioridade mais alta entra no estado Pronto**.
  - A tarefa em estado **Em Execução** entra no estado **Bloqueado** ou **Suspenso**.
- Há menos trocas de contexto de tarefas quando o fatiamento de tempo não é usado do que quando o fatiamento de tempo é usado.
  - Portanto, desativar o fatiamento de tempo resulta numa redução na sobrecarga de processamento do escalonador.
  - No entanto, desativar o fatiamento de tempo também pode fazer com que tarefas de mesma prioridade recebam quantidades muito diferentes de tempo de processamento, um cenário demonstrado.

Porquê usar um  
sistema operativo  
de tempo-real?

# Kahoot! Time



universidade de aveiro



theoria poiesis praxis